

CO3-AT2 CASE STUDY REPORT

Q32. Case Study: Sports Tournament Ranking System (Quick Sort)

Course Code: CSA0606

Course Name: Design and Analysis of Algorithms (DAA)

Name: Sharmila R

Reg. No.: 192521213

1. Student Analysis and Problem Identification

Understanding of the Case: A sports management system needs to rank teams based on their scores and performance metrics. As match results change, the ranking may need to be updated dynamically. Quick Sort can be used to efficiently arrange teams according to their scores.

Core Problem: The main problem is to sort a large number of teams efficiently while handling dynamic ranking updates. Poor pivot selection may lead to unbalanced partitions and worst-case performance.

Objectives: To apply Quick Sort to a sports tournament ranking system, understand its divide-and-conquer operation, analyze its efficiency, identify pivot-selection issues, and propose suitable improvements.

Assumptions: Each team has a score or ranking value. Higher scores represent better rankings. Each team can be represented by a record containing team name and score.

Input Requirements: Team names and their corresponding scores or performance metrics.

Output Requirements: A ranked list of teams arranged according to their scores.

Constraints: The system should handle a large number of teams efficiently. Ranking updates should require reasonable processing time. Duplicate scores may occur.

Relevant Concepts/Theories: Divide and conquer, Quick Sort, partitioning, pivot selection, recursion, time complexity, space complexity, and dynamic ranking.

2. Proposed Solution / Methodology

Selected Methodology: Use Quick Sort based on the team score. The ranking system repeatedly partitions the list around a pivot and recursively sorts the two resulting subarrays.

Algorithm / Technique: Quick Sort using a suitable pivot-selection strategy such as median-of-three to reduce the likelihood of highly unbalanced partitions.

Step-by-Step Procedure: 1. Store team names and scores. 2. Select a pivot. 3. Partition teams into lower-score and higher-score groups. 4. Recursively apply Quick Sort to both groups. 5. Combine the sorted groups to obtain the final ranking. 6. When scores change, update the affected data and perform sorting or an appropriate partial update.

Flowchart / Architecture: Start → Enter team data → Select pivot → Partition teams → Recursively sort left and right partitions → Obtain ranked list → Display ranking → Update scores when required → Re-rank → Stop.

Mathematical Formulation: Quick Sort recurrence: $T(n) = T(k) + T(n-k-1) + O(n)$, where k is the number of elements placed on one side of the pivot. Balanced partitions give approximately $O(n \log n)$ time; highly unbalanced partitions can result in $O(n^2)$.

Reason for Selecting Quick Sort: Quick Sort is suitable because it uses divide and conquer, has $O(n \log n)$ average-case time complexity, works efficiently in memory, and can be adapted for different ranking criteria. Median-of-three pivot selection can improve practical performance.

3. Implementation / Execution

Tools and Technologies Used: Python 3.x, VS Code or Google Colab, and Matplotlib for graphs.

Input Data: Example team data: Team A – 85, Team B – 72, Team C – 95, Team D – 60, Team E – 88, Team F – 78, Team G – 92.

Implementation Procedure: Represent each team with its name and score. Use Quick Sort to compare scores and partition the teams. Recursively sort the partitions until the complete ranking is obtained.

Important Code / Screenshots: Include the Quick Sort source code and a screenshot of the program execution in this section.

Intermediate Results: After each partition, the pivot is placed in its correct position and the remaining teams are divided into smaller groups. These groups are recursively processed.

Problems Encountered and Solutions: Poor pivot selection can produce highly unbalanced partitions, especially when scores are already sorted. Using median-of-three pivot selection reduces the likelihood of selecting an extreme value. For frequent updates, an efficient incremental ranking structure can also be considered.

4. Results and Analysis

Result Table: Record the actual execution time and ranking output from your implementation. Use the same input data when comparing methods.

Performance Measures: Measure execution time, number of comparisons if required, and memory usage. Compare different pivot strategies and input distributions.

Comparison with Alternative Approaches: Quick Sort generally provides $O(n \log n)$ average performance. Merge Sort also provides $O(n \log n)$ worst-case performance but requires additional memory. Insertion Sort may be simple for very small or nearly sorted lists but becomes inefficient for large datasets.

Interpretation: The ranking is obtained correctly after sorting the teams by score. Balanced pivot selection improves Quick Sort performance. Median-of-three is useful because it reduces the probability of repeatedly selecting an extreme score as the pivot.

5. Discussion

What does the result indicate?: The result indicates that Quick Sort can efficiently rank teams when the pivot produces reasonably balanced partitions. Performance depends

strongly on pivot selection and the distribution of scores.

- Did the proposed solution solve the problem?:** Yes. Quick Sort can arrange teams according to their scores and provide an efficient ranking mechanism for large datasets.
- Advantages:** Fast average-case performance, divide-and-conquer design, low auxiliary memory compared with some alternatives, and flexibility in choosing ranking criteria.
- Limitations:** Worst-case $O(n^2)$ time is possible. Recursive implementation can use significant stack space for unbalanced partitions. Frequent dynamic updates may require repeated sorting.
- Improvements:** Use median-of-three or randomized pivot selection, use an iterative implementation when appropriate, use a hybrid approach for small partitions, and consider a balanced tree or heap-based approach when updates and top-ranked queries are extremely frequent.

6. Conclusion

The case study addressed the problem of ranking sports teams based on scores and performance metrics. Quick Sort was selected because it applies the divide-and-conquer strategy and provides efficient average-case performance. The analysis showed that pivot selection is an important factor in practical performance and that poor pivot choices can lead to the $O(n^2)$ worst case. Median-of-three or randomized pivot selection can reduce the likelihood of poor partitions. The study provides an understanding of applying sorting algorithms to real-world ranking systems and highlights how algorithm selection can be improved for dynamic data. Future work can explore hybrid sorting, balanced trees, heaps, or specialized dynamic ranking structures.

Suggested Result Table

Test Case	Number of Teams	Execution Time (ms)	Result
Random Scores	1000	Actual value	Ranked successfully
Sorted Scores	1000	Actual value	Ranked successfully
Reverse Scores	1000	Actual value	Ranked successfully
Dynamic Update	1000	Actual value	Ranking updated